

Batería para máquina de tatuar: Rover R1

Documento de Software

MUSOTOKU

Javier Rodriguez
12-1-2026

ÍNDICE

1	Código Principal.....	3
1.1	Diferencias Entre Versiones.....	3
1.2	Estructura General.....	4
1.3	Descripción De Los Distintos Bloques.....	5
1.3.1	Inicio	5
1.3.2	Estado “START-UP”	9
1.3.3	Estado “WORK” (RUN / STOP):.....	10
1.3.4	Estado “SLEEP”	15
1.3.5	Estado “ERROR”	17
1.3.6	Gestor de Cambios de Estado.	18
1.3.7	Rutinas Cíclicas.	20
2	Librerías.....	21
2.1	Librerías Propias.....	21
2.1.1	BOTONERA (0001-BUTTONS)	21
2.1.2	PANTALLA LED (0002-DISPLAY)	22
2.1.3	BUZZER (0003-BUZZER)	24
2.1.4	DCDC (0004-DCDC)	26
2.1.5	SENSADO (0005-SENSING)	27
2.1.6	BARRA DE POTENCIA (0006 – POWER BAR)	30
2.1.7	LOGGING (0010-LOGGING)	31
2.2	Librerías Ajenas.....	34

1 CÓDIGO PRINCIPAL

1.1 DIFERENCIAS ENTRE VERSIONES.

- Voltaje:

Versión	Voltaje
RCA	5-12
Cheyenne	5-12
DanKubin(Low_Voltage)	2.5-12
Universal	4-12
Vantablack	4-12
Kora	2.5-12

- Modos Por defecto:

Versión	Nitro	Low Voltage	Vanta Mode
RCA	Desactivado	X	X
Cheyenne	Desactivado	X	X
DanKubin (Low_Voltage)	Activado	Activado	X
Universal	Desactivado	X	X
Vantablack	Desactivado	X	Desactivado
Kora	Desactivado	Activado	X

- Menús accesibles:

Versión	Nitro	Low Voltage	Vanta Mode
RCA	SI	NO	NO
Cheyenne	SI	NO	NO
DanKubin(Low_Voltage)	SI	NO	NO
Universal	SI	NO	NO
Vantablack	SI	NO	SI
Kora	SI	SI	NO

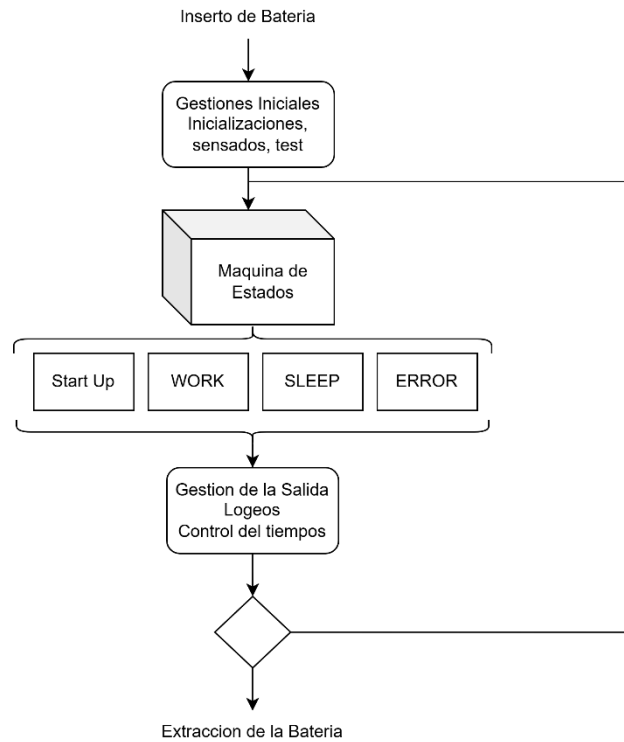
- Particularidades:

Versión			
RCA	-	-	-
Cheyenne	Cambios de Voltaje	Nitro	-
DanKubin(Low_Voltage)	-	Nitro	-
Universal	-	-	-
Vantablack	PID	Hz	Motherfucker
Kora	-	-	-

1.2 ESTRUCTURA GENERAL.

El código consta principalmente de una maquina de estados, un gestor de estados y funciones rutinarias como sensado o actuación.

El siguiente diagrama ejemplifica y resume la distribución del código:

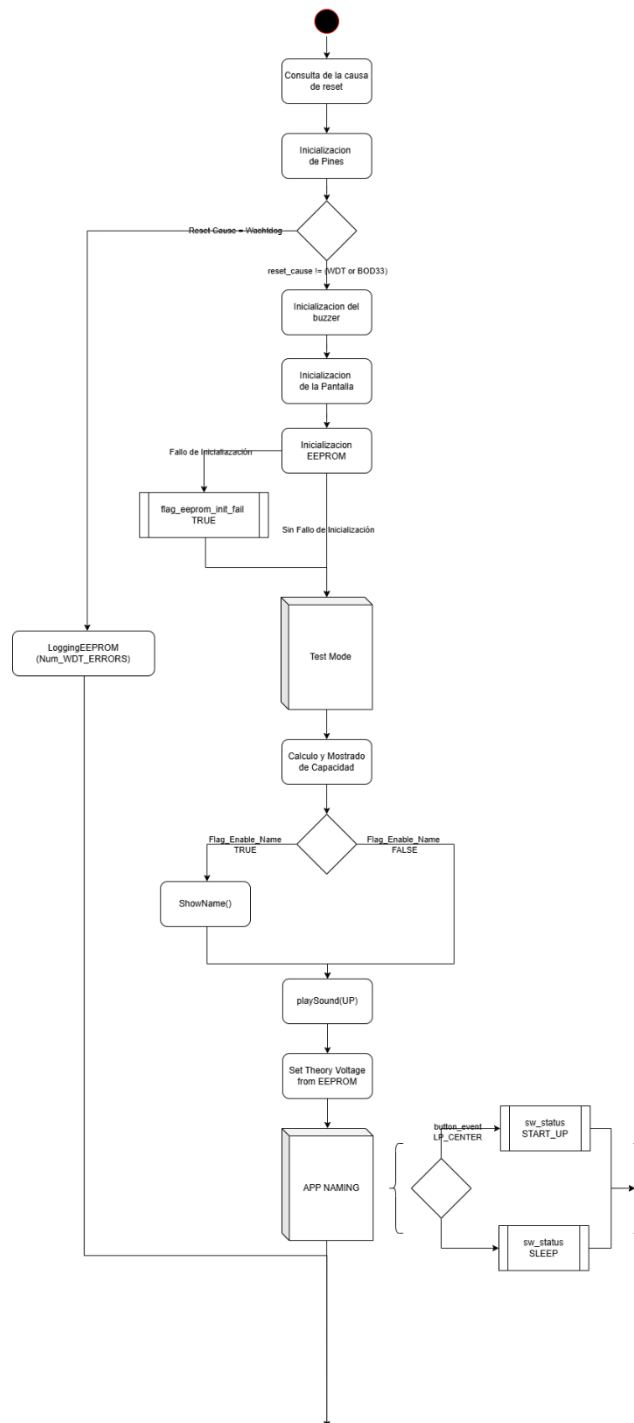


1.3 DESCRIPCIÓN DE LOS DISTINTOS BLOQUES.

1.3.1 Inicio

La inicialización o set-up, se lleva a cabo cada vez que se inserta la batería y la CPU recibe alimentación, iniciándose. Esta parte del código, previa al bucle de control, se ejecuta una sola vez.

El siguiente diagrama muestra los pasos seguidos en la secuencia de encendido:



- CAUSA DEL INICIO

En cada inicio del programa se comprueba cual ha sido la razón del reinicio. En caso de que se deba a algún comportamiento no deseado como un Watchdog o un BOD, se registra en la EEPROM antes de continuar.

NOTA: UNA DE LAS VERSIONES ANTIGUAS, TIENE UN PROBLEMA RELACIONADO CON EL WATCHDOG. EL PROGRAMA POR ALGUNA RAZÓN PIERDE EL PUNTERO, SALTANDO EL WATCHDOG Y QUEDÁNDOSE EN UN BUBLE INFINITO DE REINICIOS DE WATCHDOG. LA SOLUCIÓN ES LA REPROGRAMACIÓN.

- DECLARACIÓN E INICIALIZACIÓN DE LOS PINES:

Configuración de todos los pines usados del SAMD según su uso: Entrada o salida.
Adicionalmente se inicializan aquellos pines que son de salida.

- INICIALIZACIÓN DE LOS PERIFÉRICOS:

Algunos periféricos utilizados como el display, o el buzzer, requieren de una inicialización.

En el caso del display el controlador necesita inicializar sus registros.

En esta parte también es donde se realiza un volcado de los datos de las eeprom interna que sirven para situar el sistema en un punto de trabajo lo mas parecido al que se encontraba antes de su ultimo apagado.

En función de la versión de la Rover, se parte de una configuración de los siguientes modos:

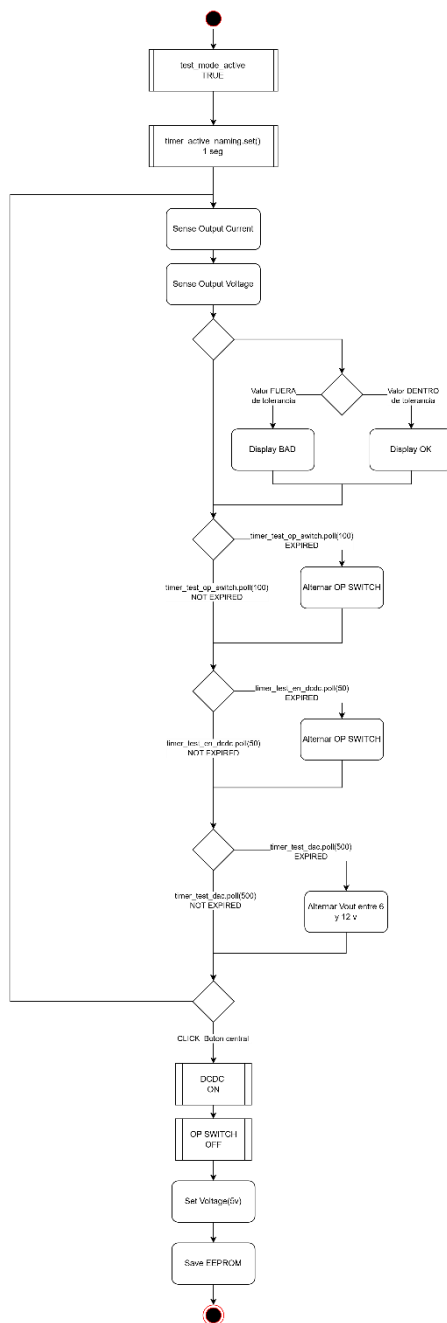
- Nitro Activado
- Low Voltage Activado
- VantaMode Activado

En el caso que alguno de los modos no se use en la versión, su valor por defecto será FALSE (Desactivado).

Versión	Nitro	Low Voltage	Vanta Mode
RCA	Desactivado	X	X
Cheyenne	Desactivado	X	X
DanKubin(Low_Voltage)	Activado	Activado	X
Universal	Desactivado	X	X
Vantablack	Desactivado	X	Desactivado
Kora	Desactivado	Activado	X

En caso de que la lectura de las eeproms no se haya producido adecuadamente se registran dichos errores en variables en RAM. Mas adelante en el ciclo de control del programa, la próxima vez que se intente acceder a las EEPROMS para actualizarlas, en su lugar se volverá a intentar la inicialización/volcado de los datos.

- TEST MODE



Cada vez que una placa es programada por primera vez, es necesario verificar que a nivel de Hardware todas las conexiones funcionan correctamente.

El test mode consta de un bucle while donde se generan una serie de señales cuadradas que se comprueban en el tester.

- OP_SWITCH
- EN_DCDC
- DAC

Además, se realiza el sensado de los pines de la corriente y voltaje para validar, a través de 2 señales de prueba, que los ADC y las etapas de acondicionamiento están correctas.

- CALCULO Y MOSTRADO DE LA CAPACIDAD:

Debido a que esta parte de código se ejecuta tras la inserción de la batería, por decisión de diseño, se realiza una medida de la capacidad de la batería insertada y dicha capacidad se muestra por el display, sirviendo como método de consulta del porcentaje de la batería para el usuario.

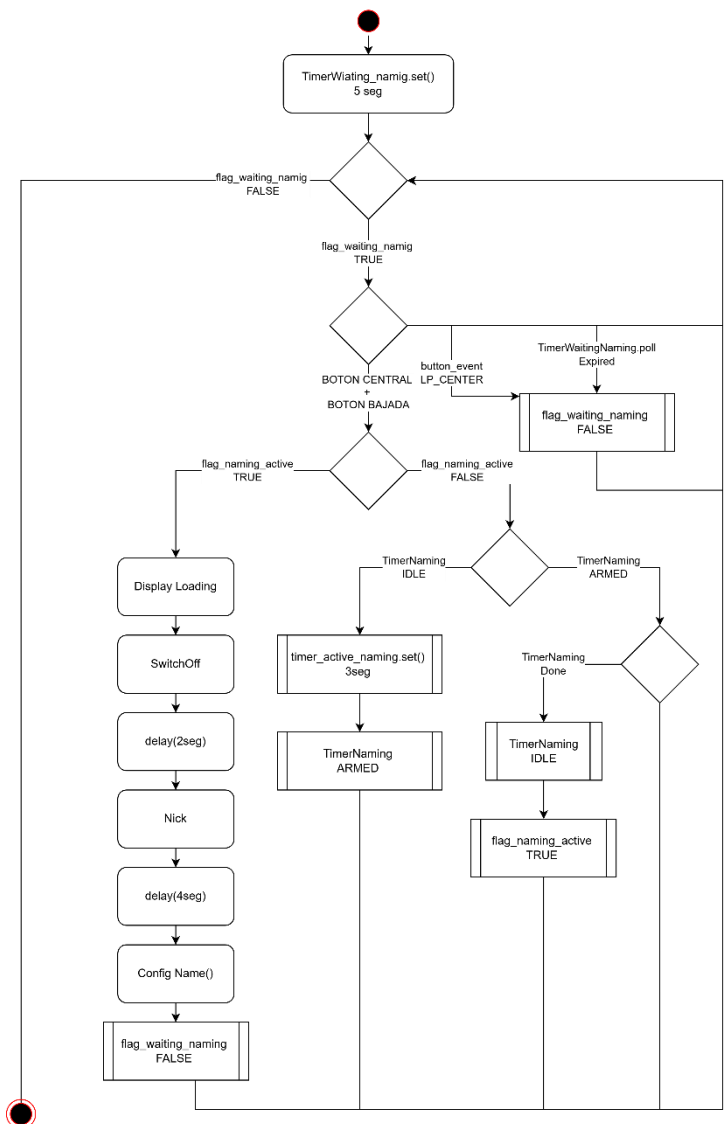
- NAMING

Una de las features presentes en la batería es la posibilidad de configurar un saludo inicial con una cadena de texto.

Si se ha realizado previamente la configuración de la cadena de texto, se mostrará en este punto.

Tras ello, la batería apaga la pantalla, y se abre la posibilidad de configurar, o reconfigurar, la cadena de texto.

1. Se abre una ventana de 5 segundos.
2. Si los botones derecho e izquierdo presionados se lanza un temporizador de 3 segundos, si durante los 3 segundos se mantienen presionados los 2 botones se accede a la configuración de la cadena de texto.
3. Si pasan los 5 segundos o se recibe una pulsación larga del botón central, se cancela la espera y se continua con el programa.
4. Tras la configuración, se almacena la información en EEPROM para que en las próximas inserciones de la batería se muestre el naming.

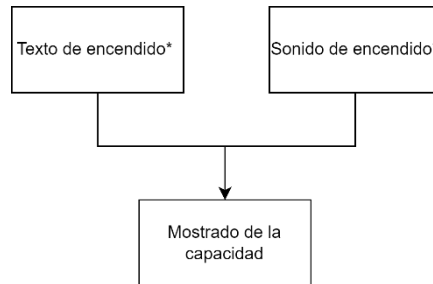


1.3.2 Estado “START-UP”

El estado de START-UP tiene lugar cada vez que se “encienda” la batería y es un estado transitorio, es decir, se ejecutará una vez y sirve de puente entre 2 estados permanentes (SLEEP y STOP).

En función de la configuración almacenada en el chasis, el proceso de encendido puede variar.

- (OPCIONAL) Se muestra un texto en el display.
- (OPCIONAL) Se escucha una música reproducida por el buzzer.
- Se muestra el porcentaje de capacidad restante de la batería.



**Es necesario que la feature se encuentre activada en la configuración interna de la batería, solo accesible por .*

- TEXTO DE ENCENDIDO

Al comienzo de la inicialización se muestra en el display un mensaje prefijado, el icono de musotoku. Por defecto, el mensaje de encendido se encuentra activado.

- SONIDO DE ENCENDIDO

Al comienzo de la inicialización se reproduce en el buzzer un sonido prefijado. Por defecto, el sonido de encendido se encuentra desactivado.

- MOSTRADO DE LA CAPACIDAD

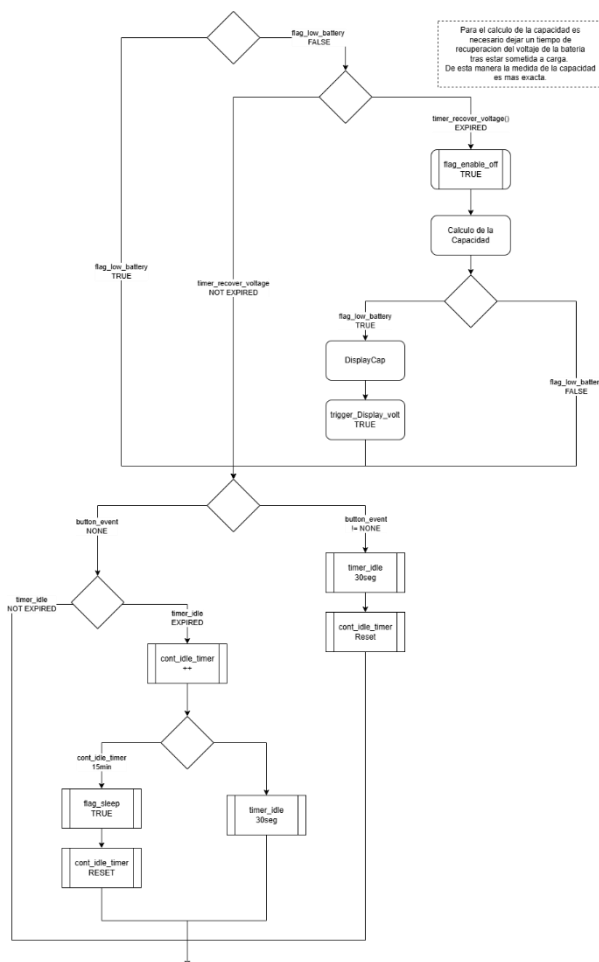
Una vez que se ha mostrado el mensaje y ha terminado el sonido. Para terminar la secuencia de encendido se muestra durante un breve tiempo la capacidad de la batería. Mientras se mantenga el botón central presionado, se mostrará la capacidad de la batería.

1.3.3 Estado “WORK” (RUN / STOP):

Dentro del estado de WORK podemos diferenciar dos “sub-estados”, RUN y STOP, que a efectos prácticos se van a tratar como estados independientes. Sin embargo, ciertas comprobaciones y actuaciones se llevan a cabo independiente del sub-estado, siempre que el sistema se encuentre en WORK.x

1.3.3.1 Estado “STOP”

El estado STOP es un estado permanente donde la alimentación de salida se encuentra desconectada.



El siguiente diagrama representa las acciones o comprobaciones que se realizan en este estado frente a distintos eventos y/o interacciones con la batería:

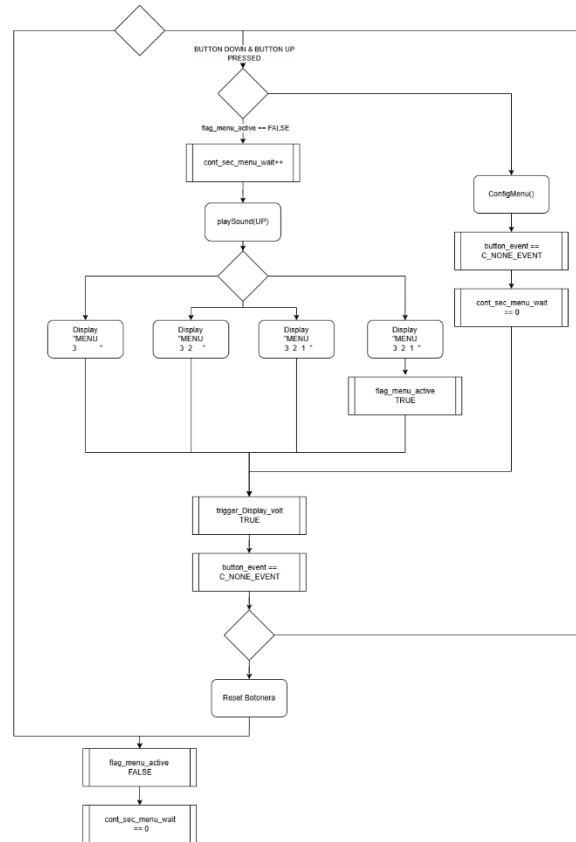
Por decisión de diseño, la actualización del valor de la capacidad de la batería se realiza durante el estado de STOP. Sin embargo, para realizar la medida hay tomar ciertas precauciones.

Debido a que la medida de la capacidad se realiza a través del voltaje observado en la batería, es necesario que este se encuentre en un estado libre de carga, por ello antes de leer se abre un abanico de tiempo en el que se espera que el voltaje de la batería se recupere en el caso de que el sistema venga del estado de RUN donde ante una carga como es una maquina de tatuar, la batería se arruga, disminuyendo su voltaje.

Una vez el timer de recuperación haya terminado, se puede realizar el cálculo de la capacidad.

Adicionalmente, en el estado de stop se monitorea si el sistema no recibe ninguna interacción durante un tiempo determinado (15 min aprox), en cuyo caso se considera que el sistema esta inactivo y se procede al apagado del mismo (SLEEP).

Durante el Estado de STOP, también se hace la atención para entrar en el Menu. Para ello se debe pulsar los botones izquierdo y derecho simultáneamente. Cada segundo que pase ira cambiando la pantalla hasta que salte el menú.



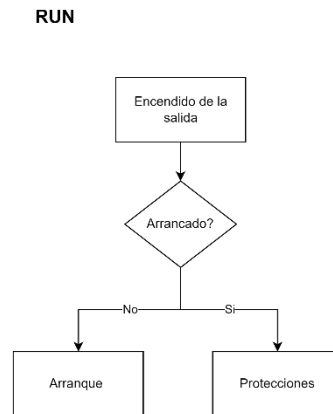
En función de la versión de la ROVER R-1 las opciones dentro del menú variaran:

Versión	Nitro	Low Voltage	Vanta Mode
RCA	SI	NO	NO
Cheyenne	SI	NO	NO
DanKubin(Low_Voltage)	SI	NO	NO
Universal	SI	NO	NO
Vantablack	SI	NO	SI
Kora	SI	SI	NO

1.3.3.2 Estado "RUN"

El estado RUN es un estado en el que se permanece hasta que el usuario decide parar la máquina y por tanto la alimentación de salida se encuentra conectada.

El siguiente diagrama representa las acciones o comprobaciones que se realizan en este estado frente a distintos eventos y/o interacciones con la batería:



El estado RUN se corresponde con la activación de la salida, esto quiere decir que es cuando alimentamos a la máquina de tatuar conectada. Por ello hay que identificar 2 escenarios: el arrancado y el régimen permanente.

Durante el arrancado, hay que tener en cuenta que estamos alimentando máquinas de tatuar que pueden demandar potencias muy altas en las fases iniciales (hasta x10 veces el consumo estacionario), por ello, puede darse la situación que nos encontremos con picos de potencia que, al llegar a la batería, su voltaje caiga llegando, en algunos casos, a niveles peligrosos para la alimentación general del sistema, es decir que puedan:

1. Alcanzar un voltaje insuficiente para el funcionamiento de la CPU
2. Alcanzar un voltaje que dispare el protector de la batería y bloquee su salida permanentemente

Para prevenir este efecto, se ha tomado la decisión de implementar una secuencia de arranque gradual más benévola con la batería que el arranque directo.

La secuencia comienza por, independientemente del voltaje de salida objetivo, durante un tiempo (milisegundos) se pone la salida a un voltaje mínimo (4-5v). Tras ello y de manera gradual se implementa una escalera de voltajes en incrementos pequeños hasta llegar al voltaje objetivo marcado. De esta manera se pretende activar las máquinas siempre a voltajes donde no tengan picos de arrancados muy brutos que no perjudiquen a la batería y por tanto el voltaje de alimentación no caiga.

➤ En el caso de la versión LowVoltage (Dan Kubin)

Cuando las máquinas funcionan por encima de 5 voltios, subir a 12v en el pico del nitro, puede significar dañar la máquina por lo tanto se implementa un nitro que sube a 4 voltios por encima del valor objetivo. Adicionalmente se suprime la existencia del micro nitro por la misma razón.

➤ En el caso de la versión Cheyenne:

NO tiene micro nitro.

El nitro no tiene rampa de subida, sino que empieza directamente en 12 durante 600ms. Tras ello tiene una rampa de bajada hasta el valor de voltaje objetivo.

Una vez, la maquina se encuentre arrancada, consideramos régimen estacionario. En este régimen existen unas protecciones de corriente, voltaje de salida, voltaje de entrada y potencia, que permitirán detectar cualquier mal funcionamiento y actuar en consecuencia.

Las protecciones funcionan con la librería HealthMonitor.h, dicha librería implementa un algoritmo de temporizadores de rampas de subida y bajada en función de la persistencia o no de los valores por encima del umbral fijado.

Protección	Threshold	T min de activacion	Inc	Dec	lim	N.º de ciclos min para activación
Cortocircuito	1.9A	500ms	10	1	200	$200/10= 20$
Consumo	1A	1500ms	10	1	600	$600/10= 60$
Potencia	5W	2000ms	10	1	800	$800/10 =80$
UnderVoltage	-1v	1000ms	1	10	400	$400/10 = 40$

1.3.3.3 Rutinas del modo Work

Procesos que se llevan a cabo tanto el estado de Run como en el de Stop:

- BOTONES:

Dependiendo de las acciones que se lleven a cabo sobre los distintos botones del sistema se producen unos cambios u otros, si se produce:

- Una pulsación corta sobre el Up-Button, se sube el voltaje 0.1v.
- Una pulsación corta sobre el Down-Button, se baja el voltaje 0.1v.
- Una pulsación larga sobre el Up-Button, se sube el voltaje 1v desde el voltaje actual (+1V).
- Una pulsación larga sobre el Down-Button, se baja el voltaje 1v desde el voltaje actual (-1V).

➤ En el caso de la versión de Cheyenne:

La subida y bajada de voltaje de voltio en voltio. Se realiza mediante escaleras para evitar cambios bruscos en la alimentación del motor que puedan provocar inestabilidad en el controlador de las máquinas de Cheyenne.

Si se mantiene pulsado cualquiera de los 2 botones, se continúa aumentando/disminuyendo el voltaje de manera repetida hasta que se suelte el botón o se alcanzan los límites permitidos.

- Una pulsación larga sobre el Power-Button, se inicia o continua la secuencia de apagado. Para pagar el sistema es necesario mantener presionado el botón durante al menos 5 pulsaciones largas. En cada pulsación se lleva a cabo una acción distinta:
 - La primera muestra la capacidad.
 - La segunda, tercera y cuarta muestra un aviso de que se va a apagar la batería progresivamente.
 - La quinta activa el cambio de estado para apagar el sistema.

Si no se completa de manera consecutiva las pulsaciones largas, el contador se resetea y se muestra de nuevo el voltaje actual.

- En el caso de la Version VANTABLACK:

Adicionalmente, si los botones derecho y central se pulsan, activamos/desactivamos el modo "MotherFucker". Este modo cambia el modo de visualización de la barra de potencia (la barra se vuelve negra y el fondo blanco).

- ACTUALIZACIÓN DE LA PANTALLA.

En el caso de que se haya modificado el valor del voltaje de salida, se haya producido un cambio de estado o se haya interrumpido la secuencia de apagado, la pantalla muestra el voltaje teórico en ese instante.

- En el caso de la versión VANTABLACK:

Si el modo MotherFucker esta activado, en vez de actualizar la pantalla el valor del voltaje, se sustituye por mostrar los HZ equivalentes.

- ACTUALIZACIÓN DE LA BARRA DE POTENCIA.

Se manda actualizar la cantidad de puntos encendidos y apagados de la barra de potencia para reflejar la medida de potencia instantánea. En el momento de pasar al estado RUN siempre se activa al menos el primero de los leds para indicar que la salida está activada. El resto de los leds se encienden en función de la potencia instantánea medida ($v_{out} * i_{out}$).

1.3.4 Estado "SLEEP".

El estado de SLEEP es un estado permanente en el que el sistema se encuentra alimentado (la batería esta insertada en el chasis, y por tanto la CPU esta funcionando) pero se considera "apagado".

Durante este estado, el objetivo es reducir al mínimo el consumo.

- TEXTO DE ENCENDIDO

Al principio de la secuencia de apagado se muestra en el display un mensaje prefijado. Por defecto, el mensaje de apagado se encuentra desactivado.

- SONIDO DE ENCENDIDO

Al principio de la secuencia de apagado se reproduce en el buzzer un sonido prefijado. Por defecto, el sonido de apagado se encuentra desactivado.

- CONTROL DE LA PULSACIÓN DEL BOTÓN CENTRAL O POWER BUTTON.

Se percibió que existía la posibilidad que si se mantenía el botón apretado se podía generar un ciclo de encendido y apagado constante.

Por ello, antes de continuar con la secuencia de apagado se realiza una comprobación continua del estado del botón hasta que este se haya dejado de pulsar.

- ENTRADA EN MODO BAJO CONSUMO.

Se desactiva la alimentación alternativa que sirve de alimentación de periféricos como el display, el dc/dc, el buzzer etc. De esta manera se reduce considerablemente el consumo general del sistema.

- CPU EN MODO SLEEP.

El SAMD tiene un modo Sleep donde el consumo se reduce drásticamente. Al entrar en este modo, el ciclo de programa se congela. Es necesario setear una interrupción de Wake-Up para despertar al procesador. Una vez el procesador se despierta reanuda la ejecución del programa en la instrucción siguiente a la que le pone a dormir.

En este caso la interrupción queda seteada al pin del botón central o Power Button. En cuanto se pulsa el botón el procesador se despierta y continua con el programa.

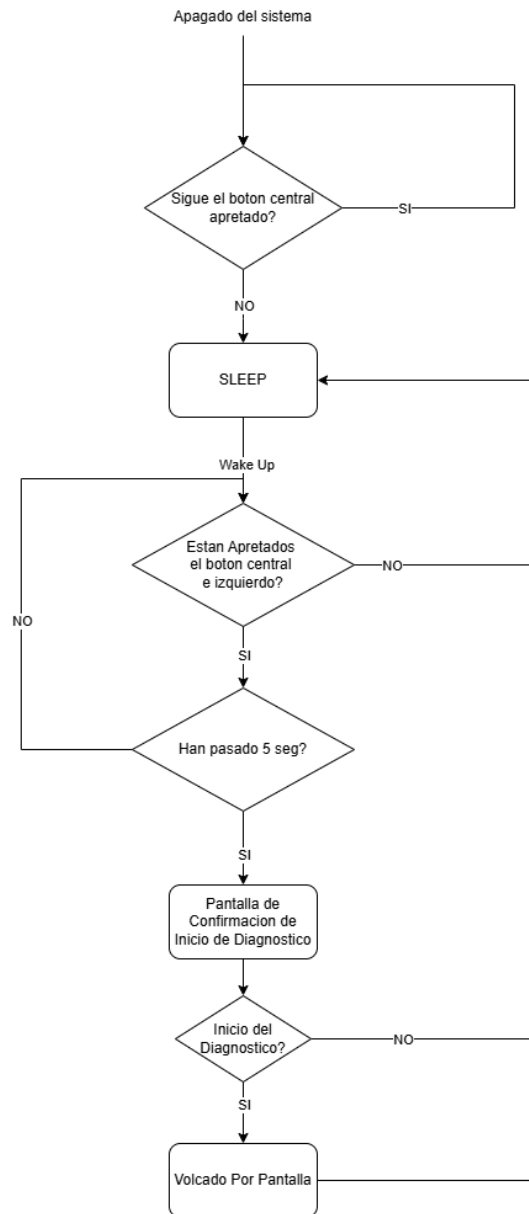
- CONTROL DE PULSACIÓN DEL BOTÓN CENTRAL.

Con el fin de prevenir activaciones involuntarias, una vez que el procesador se despierta, se procede a evaluar si el botón esta pulsado y si es así durante cuanto tiempo lleva pulsado. En este caso es necesario detectar una pulsación larga para que se entienda como una activación voluntaria. Si en algún momento se deja de pulsar el botón antes de cumplir el tiempo, el sistema volvería a iniciar la secuencia de dormir desde el guardado en eeprom para volver a meter al procesador en modo sleep o bajo consumo.

- MODULO DIAGNOSTICO

Para poder realizar una supervisión del estado de la batería a distancia, se decide incluir un modo que muestre por pantalla una serie de registros que informan sobre el uso que ha tenido la batería además de datos temporales y de los errores sufridos.

Para activarlo es necesario que desde el estado de Sleep, se mantenga pulsado el botón central y el izquierdo durante 5 segundos.

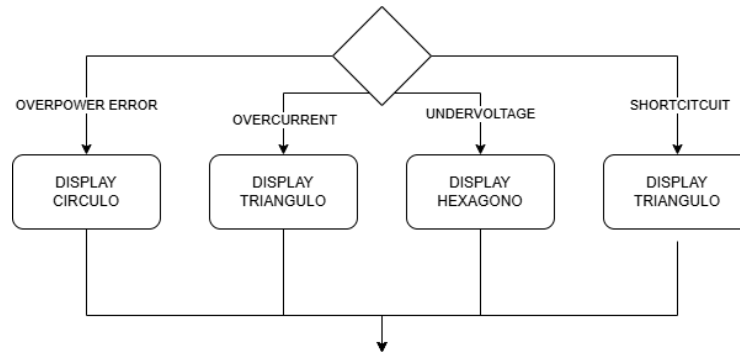


1.3.5 Estado "ERROR".

El estado de "ERROR" es un estado transitorio donde se llevan a cabo todos los procesos de atención a errores.

El estado error es un estado en el que la salida queda inmediatamente deshabilitada.

Cualquiera de las razones por las que se accede a este modo tiene que ver con la salida, por lo que es importante que se corte en cuanto se detecta un error.



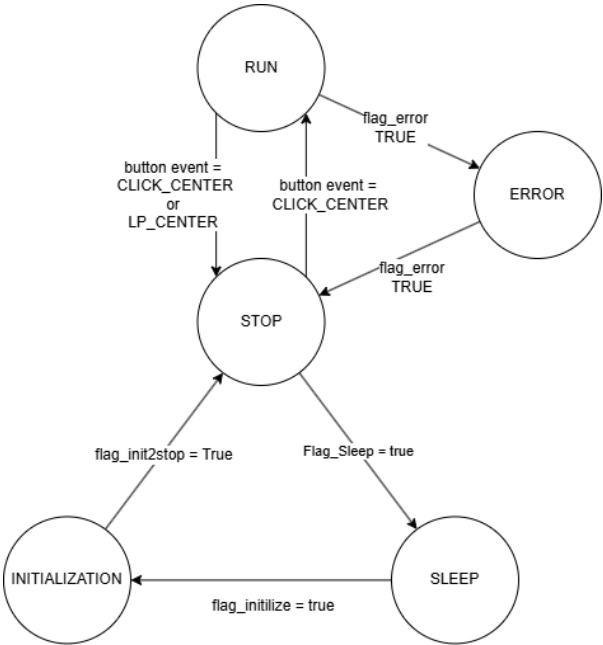
Actualmente todas las atenciones de errores pasan por mostrar un mensaje de aviso del error, de un sonido que lo acompaña y el parpadeo de la barra de potencia

Tras ello se realiza un guardado en eeprom para que quede registro de los problemas que ha sufrido el sistema.

1.3.6 Gestor de Cambios de Estado.

Este bloque es el encargado de detectar las condiciones de cambio entre los distintos estados y de ejecutar las acciones necesarias que tienen lugar en esos cambios.

El siguiente diagrama es un resumen de todos los cambios de estado enumerados:



A continuación, se enumeran las condiciones de cambio entre los distintos estados y las acciones, si las hubiera, consecuentes a dichos cambios:

ORIGEN	DESTINO	CONDICION	ACCION
START_UP	STOP	Secuencia de encendido terminada	- Inicio de timer de inactividad. - Inicio de timer de recuperación de voltaje. - Actualización de la pantalla
	ERROR		- Desactivación de la salida. - Sonido de error.
RUN	STOP	Pulsación del power button. Pulsación larga del power button	- Inicio del timer de inactividad. - Inicio del timer de recuperación. - Sonido de “desactivación de la salida”.
	ERROR	Activación de Flag de Error.	- Desactivación de la salida. - Sonido de error.
STOP	RUN	Pulsación del power button.	- Sonido de “Activación de la salida”.
	SLEEP	Secuencia de apagado finalizada.	- Inicio timer pantalla apagado

Sleep	START_UP	Despertar completado (pulsación continuada del Power button).	- Inicialización de la pantalla. - Display Logo - Inicio timer pantalla inicial
	USB	Detección de la conexión de un cargador en el puerto USB-C mediante interrupción.	- Animación de conexión USB (Imagen y sonido)
ERROR	STOP	Secuencia de atención a errores terminada.	- Inicio del timer de inactividad. - Inicio del timer de recuperación. - Actualización de pantalla

1.3.7 Rutinas Cíclicas.

En este bloque se agrupan aquellas funcionalidades que se ejecutan en cada ciclo de control y que son inherentes al estado en el que se encuentre el sistema.

- Sensado: Toma de medidas analógicas de las variables de: Tensión de salida, corriente de Salida y calculo de la Potencia de salida.
- Control Boost/No Boost: En el caso que se detecte corriente a la salida se activa el modo boost. En caso negativo, se pasa a modo no-boost.
- Control de la salida: En función de dos señales (software o usuario) se activa o desactiva el voltaje de salida.

➤ En el caso de la versión de VantaBlack:

Antes de generar la tensión de salida, se aplica un pid que intenta mantener la velocidad correspondiente al voltaje fijado por el usuario.

- Logging: Guardado de las variables dinámicas de las variables de diagnóstico y comprobación del timer que actualiza la eeprom.
- Lectura de los botones: comprobación de si ha ocurrido algún evento en los botones del sistema
- Control del periodo del ciclo de programa: Control del periodo mínimo del ciclo de programa, 25 milisegundos.

2 LIBRERÍAS

2.1 LIBRERÍAS PROPIAS

2.1.1 BOTONERA (0001-BUTTONS)

El sistema tiene 3 botones sobre los que se puede actuar.

La función de estos botones será: Control del voltaje de salida, activación y desactivación de la salida de voltaje y Apagado y encendido de la batería.

- Dpad.h

Esta librería contiene las funciones necesarias para la detección de pulsaciones cortas (clicks) y pulsaciones largas (Longpress) sobre cualquiera de los 3 botones del sistema

- ReadDirPad ():

Esta función comprueba el estado de las señales que indican el estado de los pulsadores. Devuelve una variable que indica el evento que ha tenido lugar, pudiendo ser ninguno.

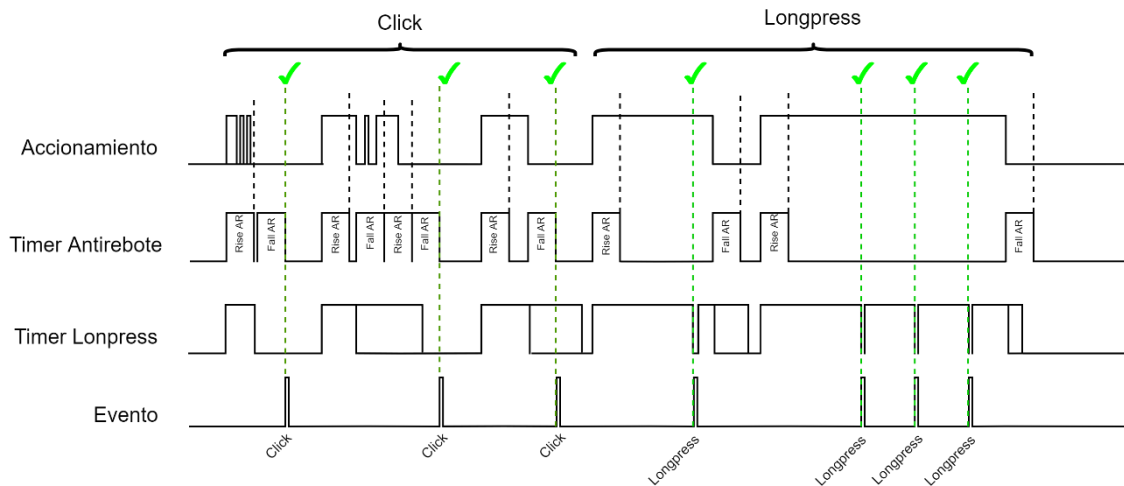
Una dinámica que se suele presentar en los pulsadores son los rebotes, por lo que es necesario implementar un temporizador que filtre estos rebotes. Durante el tiempo que esta activo el temporizador se ignorara cualquier cambio que se produzca en la línea del pulsador. Este temporizador se activará tanto en la detección de los flancos de subida como los de bajada.

- Click:
 1. Se detecta el flanco de subida
 2. Inicio del timer de anti-rebote y del timer que detecta un Longpress
 3. Tras finalizar el timer de anti-rebote, se detecta un flanco de bajada antes de que el timer de longpress haya terminado.
 4. Inicio del timer de anti-rebote.
 5. Finaliza el timer de anti-rebote.
 6. Se da el aviso de que se ha dado un Click event.
- Longpress:
 1. Se detecta el flanco de subida
 2. Inicio del timer de anti-rebote y del timer que detecta un Longpress.
 3. Finaliza el timer anti-rebote.
 4. Finaliza el timer de Longpress.
 5. Se actualiza el evento a evento de Longpress del boton asociado.
 6. Inicio del timer de longpress con otro tiempo más corto.
 7. Finaliza el timer de Longpress.
 8. Se actualiza el evento a evento de Longpress del boton asociado.

Se repite los eventos 6-7-8 hasta que se detecte un flanco de bajada.

Los eventos son One-Shot, es decir, solo se devuelven de la función en el momento que ocurren, el resto de tiempo la función devuelve NO_EVENT.

En el siguiente diagrama de tiempo se puede observar el funcionamiento y detección de los eventos de click y longpress en cualquiera de los 3 botones.



Los tiempos establecidos para los timers son los siguientes:

- Anti-rebote: 50ms.
- Longpress: 1000 ms + T anti-rebote.
- Longpress corto: 500ms

2.1.2 PANTALLA LED (0002-DISPLAY)

Display led de 0,49(64x32 px)". Este display se "parte" en dos, de manera que queda una pantalla que se utiliza para la interfaz de la batería con el uso de caracteres ASCII y/o bitmaps, y además queda una "barra" de leds.

La comunicación con el integrado es SPI y existen previamente librerías que permiten escribir caracteres y bitmaps. Por ello esta librería actúa de wrapper de esas librerías y construye las herramientas necesarias para el sistema.

- Bitmap.h

En este archivo, se encuentran todos los bitmaps necesarios usados para la visualización en el display.

Entre estos bitmaps se encuentran:

- Caracteres ASCII.
- Bitmaps de carga.
- Bitmaps de error.
- Bitmaps que actualmente no se encuentran en uso.

Además de los bitmaps, en el .h se encuentra la función *Char2Bitmap()* que recibe como argumento un carácter y devuelve el bitmap correspondiente a ese carácter.

- `Display.h`

Esta librería agrupa las principales funciones de control de la visualización en el display.

- `initDisplay()`

Inicialización de la pantalla a través del puerto SPI.

- `DisplayLogo()`

Logo de Musotoku.

- `SwitchScreenOff()` / `ScreenON()`

Apagado / Encendido de la pantalla.

- `DisplayEndScreen()`

Muestra de la pantalla de apagado de la batería en el modo Sleep.

- `DisplayVolt(volt)`

Esta función imprime la pantalla de voltaje, que muestra el voltaje al que se encuentra la salida de la batería, con un decimal.

- `DisplayCap(capacity)`

Esta función muestra la pantalla de porcentaje de capacidad de batería.

- `DisplayLowBattery()`

Pantalla de aviso de batería baja.

- `DisplayEmptyBattery()`

Pantalla de aviso de que la batería está prácticamente agotada.

- `DisplayArray()`

Función que imprime por pantalla los valores, en Hexadecimal, de un array. Esta función se usa en el modo diagnóstico para volcar los datos internos.

- `DebugDisplay()`

Composición de pantalla que permite imprimir por pantalla los valores de corriente, voltaje y potencia.

- `DisplayHz()`

Muestra la velocidad de la máquina en HZ. En función de la configuración los cálculos de velocidad se corresponden con un motor u otro.

```
vmotor = (float)voltage / 10 - 8.2 * (float)current / 1000; // Motor de 6v
```

hertz = vmotor * 18.51 + 20; // @6v 18.51 es el valor de la frecuencia en Hz por cada voltio de la batería.

2.1.3 BUZZER (0003-BUZZER)

El sistema cuenta con un zumbador(buzzer), que actúa como indicador/confirmador sonoro de las interacciones que se tienen sobre la batería.

A través de la librería nativa de Arduino de *tone* se generan las distintas notas.

- `tone(pin, frecuencia,[opcional] duracion)`

Genera una onda cuadrada de la frecuencia especificada (y un ciclo de trabajo del 50%) en un pin. Se puede especificar una duración, de lo contrario la onda continúa hasta una llamada a `noTone()`. El pin puede conectarse a un zumbador piezoeléctrico u otro altavoz para reproducir tonos.

Sólo se puede generar un tono a la vez. Si ya se está reproduciendo un tono en un pin diferente, la llamada a `tone()` no tendrá efecto. Si el tono se está reproduciendo en el mismo pin, la llamada establecerá su frecuencia.

- `Notes.h`

Archivo que contiene una LUT con las frecuencias asociadas a cada nota musical.

- `Buzzer.h`

Para poder generar un efecto de sonido (SF) es necesario crear una “melodía” con las notas y los tiempos. Cada melodía se graba en un array con el siguiente formato:

`MELODIA[] = {NOTA, tiempo de reproducción, espacio entre notas, NOTA, tiempo de rep...}`

Las melodías del sistema son:

- Sonido de Encendido.
- Sonido de Apagado
- Sonido de Bajada de voltaje.
- Sonido de Subida de voltaje.
- Sonido de Activación de la Salida.
- Sonido de Desactivación de la Salida.
- Sonido de Conexión de USB. (Deprecated)
- Sonido de Desconexión de USB. (Deprecated)
- Sonido de Error.
- Sonido de Batería baja.

- `InitBuzzer(pin,mode)`

Función que configura el pin asociado al buzzer y configura los sonidos del sistema. Es posible que existan varios modos (o grupos de melodías) del sistema, en función del argumento *mode*, se seleccionan un grupo de melodías u otras. Existen un modo por defecto en el caso que no se especifique en el argumento.

- *playsound(num)*

Función encargada de reproducir la melodía seleccionada por el argumento *num*.

1. La función copia en una variable local el array correspondiente a la melodía. (función *getSound()*)

De manera bloqueante la función recorre el array.

2. En cada iteración reproduce el sonido usando la función *tone()* con la frecuencia y el tiempo de reproducción como argumentos.
3. Se lanza un timer con la duración de la nota más el tiempo entre notas.
4. Una vez finaliza el timer, se continua con la siguiente nota.

Este proceso se repite hasta que se reproducen todas las notas de la melodía.

- *getSound(sound_id)*

Esta función devuelve un array con la melodía correspondiente al *sound_id*.

- DCDC.h

Es librería implementa todas las funcionalidades necesarias en relación con el control del voltaje de salida actuando sobre el integrado TPIC2810, mediante comunicación I2C, y por tanto sobre el divisor resistivo de la realimentación del DCDC.

Se declara una clase *dcdc_controler* y se crean los siguientes métodos asociados:

- *dcdc_controler(pin)*

Creación de un objeto del tipo dcdc controler. Cada objeto estará liga a un DCDC cuyo enable se encuentra en el pin.

- *EnableDCDC(bool)*

Permite activar o desactivar el enable del DCDC.

- *SetVoltage (volt, mode)*

Configuración del valor de la señal analógica generada.

Si el valor del voltaje a generar es mayor o igual a 4.0v la señal se genera con el DAC.

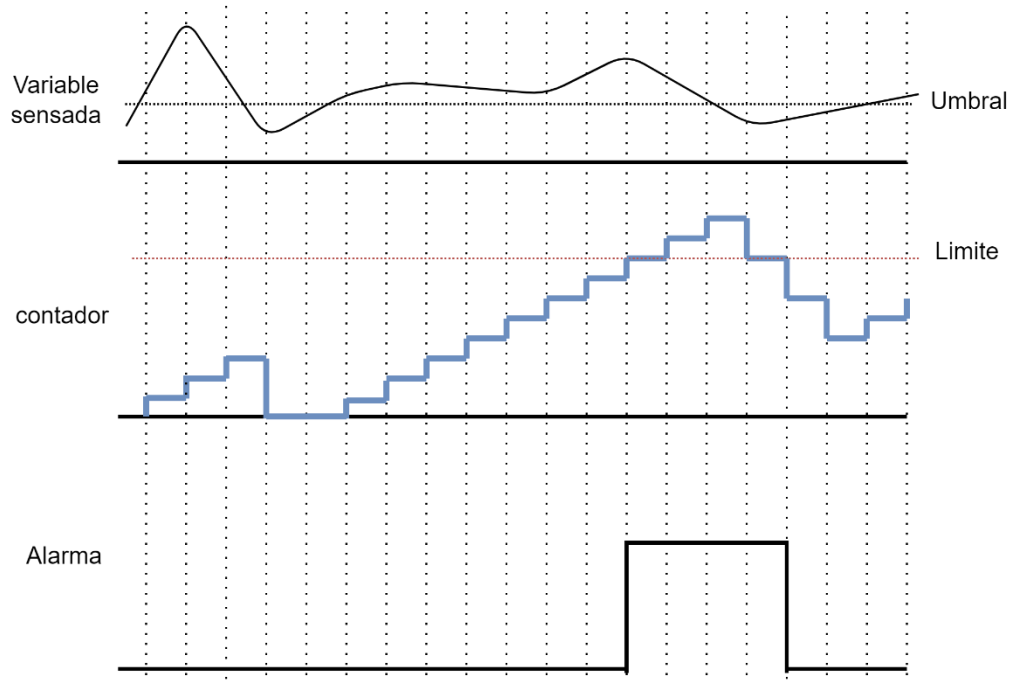
Si el valor del voltaje a generar es menor a 4.0v se realiza una señal de PWM con el tranasitor de salida. La señal tiene de amplitud de 5v, ciclo de trabajo del 50% y la frecuencia variará según el voltaje de salida deseado.

2.1.5 SENSADO (0005-SENSING)

El sistema implementa un algoritmo propio para la gestión de las protecciones. Este algoritmo consiste en la creación de rampas de subida o bajada en función de si la variable sensada se encuentra por encima o por debajo de un umbral.

Cada vez que se realiza una comprobación de la variable sensada, y esta se encuentre por encima del umbral, un contador se incrementara una cantidad determinada. Mientras que siempre que se realice la comprobación y la variable sensada se encuentre por debajo del umbral, el mismo contador se decrementa una cantidad.

Si el contador alcanza un valor limite, el algoritmo da la alarma para que se puedan tomar las acciones correspondientes.



De esta manera se crea una protección que se protege frente a momento puntuales que provocarían el salto de la alarma de funcionar simplemente como umbral.

Los parámetros configurables de este algoritmo son:

- Umbral.
- Incremento.
- Decremento.
- Límite de cuenta.

- HealthMonitor.h

Se declara una clase de *HealthMonitor* y se crean los siguientes métodos:

- *HealthMonitor (threshold, inc, dec, lim).*

Creador de la clase con las variables que fijan el umbral de comparación, el incremento por cada vez que se encuentre por encima, el decremento por cada vez que se encuentre por debajo del umbral y el límite de cuenta.

A la hora de crear el *HealthMonitor* es muy importante tener en cuenta el tiempo de muestreo.

Como aproximación se utiliza la siguiente función:

$$\text{Tiempo minimo de salto de alarma} = \frac{\text{Limite de cuenta}}{\text{Incremento}} * \text{Tiempo de muestreo}$$

El decremento marcara la independencia a los picos que pueda sufrir la señal sensada.

- *setCounter ()*

Existe la posibilidad de tras saltar una alarma se quiera resetear el contador del HealthMonitor, o simplemente fijar un valor de inicio.

Esta función permite dar un valor al contador.

- *getCounter ()*

Devuelve el valor del contador del HealthMonitor en ese instante.

- *Check(sample)*

función que realiza la comparación del *sample* con el umbral, e incrementa/decrementa el contador en función del resultado.

Si el contador alcanza el límite, esta función devuelve un aviso de alarma.

- *GetSample (ADCpin)*

Cada vez que se realiza una muestra se hace la media de dicha muestra con los 8 valores almacenados.

2.1.6 BARRA DE POTENCIA (0006 – POWER BAR)

La barra de potencia consiste en la una fila de 15 leds que se usa como medidor visual del consumo/potencia instantánea de la batería.

El objetivo de la “barra de potencia” es la representación visual de la dinámica de la maquina de tatuar durante su funcionamiento.

Dentro de la “barra de potencia” se usa el primer led como identificador de la salida. Si la salida se encuentra activa, el led se enciende, mientras que, si la salida está desactivada, el led se apaga.

- `Power_bar.h`

Además de funciones que interactúan con los leds de la barra de potencia, este fichero almacena las características de la dinámica de la barra de potencia como:

- Potencia máxima visualizada: 2.5KW.
- Posición del led que representa la salida: 1
- Potencia representada por cada Led: $2.5KW/15 = 0.167W$

Para indicar si el pid esta activado se invierte los colores de la barra de potencia por ello es necesario saber el estado que esta esta función.

- `PowerBar(leds,pid_status = false)`

Esta función enciende, de derecha a izquierda, una cantidad *leds* de pixeles y apaga el resto de los pixeles de la barra de potencia.

- `UpdatePowerBar(power sample)`

Calcula la cantidad de pixels que tiene que haber encendidos en la barra de potencia en función del valor de potencia que se pasa como argumento de la función. Tras el cálculo, actualiza la barra de potencia.

- `LedWork(State_Led)`

Función que permite encender y apagar el pixel que representa el estado de la salida del voltaje.

2.1.7 LOGGING (0010-LOGGING)

Por decisión de diseño, al tratarse de un sistema de tiempo real, es necesario implementar alguna funcionalidad de seguimiento de aquellas variables que permitan definir 2 cosas:

- Punto de trabajo del sistema en tiempo real.
- Estado del sistema con el paso del tiempo.

Debido a la dinámica del sistema (un sistema que pierde alimentación y por tanto resetea las variables en el inicio) es necesario tener 2 tipos de almacenamiento: RAM y ROM.

El almacenamiento RAM estará destinado a almacenar aquellas variables ligadas al ciclo de programa actual. (Voltaje instantáneo, estado de la salida, etc)

El almacenamiento Rom estará destinado a almacenar aquellas variables que tienen información relevante de manera permanente (núm. errores, valor del voltaje antes del apagado, etc).

El micro-controlador SAMD21G18 tendremos una memoria de 256Kbytes. La memoria de programa esta dividida en columnas (rows) y paginas (pages). Cada columna contiene 4 páginas. El tamaño de estas páginas está predefinidas a 64 bytes.

Si deseamos almacenar un tamaño de 32 bits (4 bytes) por variable, tenemos que en cada página podemos almacenar 16 variables. Sin embargo cada pagina tiene una cabecera que ocupa 4 bytes, por lo que se almacenan 15 variables.

Sin embargo, el SAMD21G18 no posee una memoria no volátil dedicada al usuario. Es necesario configurarla de la parte de memoria Flash. Por decisión de diseño se decide generar un espacio de 256 bytes, es decir, 4 paginas de 64 bytes, lo que hace un total de 60 variables almacenables.

Estas variables se almacenan en la parte no volatil de la memoria. Sin embargo, esta memoria tiene una vida útil limitada (10.000 escrituras), por lo que es necesario que se controle muy bien el momento de la escritura.

La emulación de la EEPROM en el SAMD se realiza con una función externa. El almacenamiento de los datos se realiza en un array de datos de tamaño 16bits por dato. Es necesario tener funciones destinadas al indexado de los datos en los procesos de lectura/escritura.

- EEPROM_LUT.h

Archivo donde se almacena las posiciones de las eeprom.

- EEPROMBitbang.h (DEPRECATED)

Librería de las funciones que implementan la comunicación I2C por bitbang y las funciones de fragmentación y desfragmentación de los datos de la batería.

- Diagnostic.h
 - *Init_local_eeprom (vanta_mode_status_default, nitro_state_default, low_volt_status_default)*

Inicialización de las variables de la EEPROM. En caso de que la EEPROM ya hubiera sido inicializada se realiza una lectura y un volcado del valor en EEPROM de cada variable. Se calcula el checksum para validar que no haya habido corrupción en la memoria.

- *LogDiagnosticData(data,address)*

Actualizar el valor en Ram de una variable de diagnóstico.

- *ReadDiagnosticData(address)*

Lectura del valor de un elemento de diagnóstico.

- *IncrementDiagnosticData(in,address)*

Incrementar el valor de una variable de diagnóstico.

- *SaveEeprom ()*

Tras calcular el valor del checksum con el valor de las variables de la EEPROM en RAM, se hace un volcado en la ROM.

- *PostMortemLog ()*

Con el objetivo de conocer el estado de la batería en los últimos momentos de uso se almacenan varios arrays con valores de potencia, voltaje, batería y errores. Esta función desplaza lo valores anteriores y graba en la última posición el valor actual.

- *DiagnosticMode()*

Generación de un único array con todos los datos almacenados en la EEPROM que sirva para ser mostrados por pantalla.

- *SaveNameEEPROM()*

Registro en EEPROM del array resultado de la función NAMING.

- *ReadNameEEPROM()*

Lectura del array almacenado en EEPROM del NAMING.

2.2 LIBRERIAS AJENAS

- FlashStorage.(0012-batt_FlashStorage)

Esta librería permite la emulación de una porción de memoria EEPROM sobre la memoria Flash del SAMD.

- Adafruit_SSD1306_Library (0016-Adafruit_SSD1303-049) + Adafruit_GFX_LIBRARY (0008-Adafruit_GFX_Library)

Las librerías de adafruit que proporcionan las clases y métodos que actúan de wrapper sobre la comunicación I2C del controlador de la pantalla led.

- MilliTTimer (0014-MilliTTimer)

Creación de temporizadores no bloqueantes.

- SAMD_AnalogCorrection (0013-SAMD_AnalogCorection)

Librería que suministra funciones para el calibrado del ADC del SAMD.

- SlowSoftI2CMaster (0007-SlowSoftI2CMaster)

Librería que suministra las funciones de nivel bajo de comunicación I2C utilizadas en las librerías de BitBang.

- ArduinoLowPower (0009-LOWPOWER) + RTCZero (0015-RTCZero)

Conjunto de librerías de bajo consumo que suministran las funciones que permiten inducir al SAMD en modo Sleep y despertarlo.

- Watchdog (0019-Adafruit_SleepyDog-master)

Librería que permite leer el registro del reinicio del sistema y saber la causa del inicio del sistema.

- PWM (0020-Arduno_SAMD21_turbo_PWM)

Librería que comprende las funciones que permite generar señales pwm en alguno de los pines de la CPU.